School of Computer Science and Engineering

RV University

Computer Vision Hackathon 2026

Hackathon Technical Solution Report

VISIONARY: INTELLIGENT FINE-GRAINED VEHICLE RECOGNITION AND ANALYSIS SYSTEM

Problem Statement P5 — Intelligent Fine-Grained Vehicle Recognition and Analysis System

Submitted by

[Registered Team Name] · Team ID: T20

CHAKRADHAR K (1RUA24CSE0214)
SREERAM LAGISETTY (1RUA24CSE0219)
NAGARJUNA VV (1RUA24CSE0271)
MANOJ R (1RUA24CSE0241)
SNEHAN (1RUA24CSE0232)

Guided by

[GUIDE NAME]

[Designation], School of Computer Science and Engineering

RV University

RV Vidyaniketan, 8th Mile, Mysore Road, Bengaluru, Karnataka, India – 560059

November 2026

CERTIFICATE

This is to certify that the project work titled **"Visionary: Intelligent Fine-Grained Vehicle Recognition And Analysis System"** is performed by Chakradhar K (1RUA24CSE0214), Sreeram Lagisetty (1RUA24CSE0219), Nagarjuna VV (1RUA24CSE0271), Manoj R (1RUA24CSE0241), Snehan (1RUA24CSE0232), bonafide students of [Name of the degree] at the School of Computer Science and Engineering, RV University, Bengaluru, as part of the Computer Vision Hackathon 2026 during the academic year [2026–2027].

[Guide Name]	**[Program Director]**	**[Dean]**
Guide	Program Director	Dean
SoCSE, RV University	SoCSE, RV University	SoCSE, RV University
Date:	Date:	Date:

Name of the Examiner	**Signature of the Examiner**
1.	1.
2.	2.

DECLARATION

We hereby declare that the work entitled **"Visionary: Intelligent Fine-Grained Vehicle Recognition And Analysis System"**, submitted for the Computer Vision Hackathon 2026 at RV University, is a record of original work carried out by our team. All results reported in this document were produced by the code in the project repository and can be reproduced from it. Pretrained models, datasets and libraries used are acknowledged in the References.

We further declare that the work reported here has not been submitted, in part or in full, for any other degree, diploma or competition.

Place: Bengaluru

Date: [DD/MM/YYYY]

Chakradhar K (1RUA24CSE0214)	[Signature]
Sreeram Lagisetty (1RUA24CSE0219)	[Signature]
Nagarjuna VV (1RUA24CSE0271)	[Signature]
Manoj R (1RUA24CSE0241)	[Signature]
Snehan (1RUA24CSE0232)	[Signature]

ABSTRACT

Fine-grained vehicle recognition — naming a car's make, model and year — is hard because many of the 196 Stanford Cars classes differ only by small details such as a facelifted bumper, a badge or a roof, while real photographs vary in lighting, viewpoint, resolution and occlusion. This report presents **Visionary**, a hybrid system that combines classical computer vision with deep learning. A YOLOv8n detector localises every vehicle; an adaptive illumination gate measures each crop in CIE-LAB space and applies CLAHE only to dark or flat images; a ConvNeXt classifier, fine-tuned on detector-like crops and calibrated with temperature scaling, predicts the class with test-time flip averaging; and a classical HOG + LBP + HSV classifier provides the required baseline and an optional fusion branch whose weight is tuned on a held-out validation split.

On the official test split of 8,041 images that were never used for training or tuning, the hybrid system reaches **94.73 % Top-1 and 99.3 % Top-5 accuracy** (ConvNeXt-Small at 320 px), compared with 94.03 % for deep learning alone and 19.4 % for classical features alone. Detection achieves a mean IoU of 0.93 and mAP@50 of 0.78; temperature scaling reduces the expected calibration error from 16.8 % to 1.9 %. Dynamic INT8 quantisation shrinks the classifier 3.5× with no measurable accuracy loss. The complete pipeline was ported to the browser (ONNX Runtime Web + OpenCV.js), verified against the Python implementation, and deployed free of cost as a static web application that handles multiple vehicles per image, low-confidence inputs and non-car images.

Keywords: fine-grained recognition, vehicle classification, YOLOv8, ConvNeXt, CLAHE, hybrid computer vision, model calibration, INT8 quantisation, ONNX Runtime Web.

ACKNOWLEDGEMENT

We express our sincere gratitude to [Guide Name], [Designation], School of Computer Science and Engineering, RV University, for guidance and encouragement throughout this work.

We thank [Dean / Program Director names], School of Computer Science and Engineering, RV University, and the organisers of the Computer Vision Hackathon 2026 for the opportunity and the problem statements.

We also acknowledge the creators of the Stanford Cars dataset, and the open-source communities behind PyTorch, timm, Ultralytics YOLO, OpenCV, ONNX Runtime and Hugging Face, whose tools made this work possible.

Place: Bengaluru

Date: [DD/MM/YYYY]

Team T20

CONTENTS

CERTIFICATE i

DECLARATION ii

ABSTRACT iii

ACKNOWLEDGEMENT iv

CONTENTS v

LIST OF FIGURES vii

LIST OF TABLES vii

Project Details 1

1. Problem Understanding & Solution Strategy 2

    1.1 Core Technical Challenge 2

    1.2 Proposed Methodology 2

2. Technical Architecture & Implementation 3

    2.1 Classical Algorithmic Baseline 3

    2.2 Deep Learning Pipeline 4

    2.3 Hybrid Integration & Adaptive Mechanism 5

3. Experimental Benchmarks & Evaluation 6

    3.1 Quantitative Benchmarks 6

    3.2 Visual Outputs & Intermediate Representations 8

    3.3 Ablation Study 9

4. Failure Modes & Limitations 9

    4.1 Observed Failure Cases 10

    4.2 Root Cause & Mitigation 10

    4.3 Limitations and Ethics 11

5. Deployment & Quickstart Guide 11

    5.1 Quickstart Commands 11

    5.2 Deployment Format 11

Student Submission Checklist 12

References 12

LIST OF FIGURES

Figure 1: End-to-end architecture of Visionary 3
Figure 2: Top-1 accuracy of classical, deep-learning and hybrid approaches for both model variants 7
Figure 3: Robustness under seven synthetic corruptions (Pro model) 7
Figure 4: Confidence threshold trade-off between coverage and accuracy 8
Figure 5: Outputs on unseen test images — single vehicle, dark image (CLAHE applied), several vehicles 8
Figure 6: Intermediate states for the dark image: ROI, CLAHE, bilateral filter, Canny edges, contours 8
Figure 7: Observed failure cases: facelift year, roof-up convertible, same-maker sedan 10

LIST OF TABLES

Table 1: Project and submission details 1
Table 2: Team members and responsibilities 1
Table 3: Classical vs deep learning vs hybrid on the test set (8,041 images) 6
Table 4: Localisation, detection, calibration and implementation-parity metrics 6
Table 5: Ablation study on the test set (Top-1 %) 9
Table 6: Most frequent confusions of the Pro model on the test set 9

PROJECT DETAILS

Table 1: Project and submission details

Field	Details
Problem ID	P5
Problem Title	Intelligent Fine-Grained Vehicle Recognition and Analysis System
Team Name	[Registered Team Name] (Team ID: T20)
Team Members	Chakradhar K (1RUA24CSE0214); Sreeram Lagisetty (1RUA24CSE0219); Nagarjuna VV (1RUA24CSE0271); Manoj R (1RUA24CSE0241); Snehan (1RUA24CSE0232)
GitHub Repository	https://github.com/chakradhar1778/Visionary — commit [commit hash evaluated]
Live Demo / Deployment	https://visionalpha-visionary.static.hf.space (Hugging Face static Space; in-browser inference)

Table 2: Team members and responsibilities

Member	USN	Responsibility
Chakradhar K	1RUA24CSE0214	Deep learning & training
Sreeram Lagisetty	1RUA24CSE0219	Team lead, integration & deployment
Nagarjuna VV	1RUA24CSE0271	Classical computer vision
Manoj R	1RUA24CSE0241	Detection & localisation
Snehan	1RUA24CSE0232	Optimisation, evaluation & failure analysis

1. PROBLEM UNDERSTANDING & SOLUTION STRATEGY

1.1 CORE TECHNICAL CHALLENGE

The task is to detect vehicles in real-world photographs and name the **make, model and manufacturing year** of each one, choosing among the 196 classes of the Stanford Cars dataset, and to display a bounding box, a confidence score and the intermediate image-processing results.

The primary technical barrier is **inter-class similarity at a fine grain**: many classes share the same body shell and differ only by a facelift (e.g. Bentley Continental GT 2007 vs 2012), a trim or badge (Audi A5 vs S5), or a roof (coupe vs convertible). With only about 41 training images per class, a model must learn subtle part-level cues. The failure modes we targeted are:

- **Look-alike classes** — facelift years, trims and body-style variants;
- **Imperfect localisation** — background clutter and other vehicles in the frame;
- **Illumination and image quality** — night/dark photographs, over-exposure, motion blur, sensor noise and low resolution;
- **Partial occlusion and unusual viewpoints;**
- **Out-of-distribution inputs** — cars outside the 196 classes and images with no vehicle at all, which must not produce confident wrong answers.

Target constraints: a public URL that responds within about 10 seconds on a phone or laptop, robust handling of bad input, zero hosting cost, and a model small enough to run without a GPU.

1.2 PROPOSED METHODOLOGY

Visionary first **localises** every vehicle with YOLOv8n, then **adapts the input** to each crop: it measures brightness and contrast in CIE-LAB space and applies CLAHE only when the crop is dark or flat, instead of a fixed global enhancement. A **fine-tuned, temperature-calibrated ConvNeXt** classifies each crop (with a horizontal-flip second view), a classical descriptor classifier provides a baseline and a validation-tuned fusion branch, and a **confidence gate** decides whether to answer, warn or report "no vehicle".

This adaptive design was chosen over static baselines because a fixed pipeline either over-processes good images (always-on CLAHE changes the statistics the network was trained on) or under-serves difficult ones. Cropping to the detected vehicle removes background clutter and gives the classifier more pixels on the discriminative parts, which the ablation shows is the single largest gain.

2. TECHNICAL ARCHITECTURE & IMPLEMENTATION

Figure 1 shows the end-to-end pipeline. The same pipeline is implemented twice: `src/hybrid/engine.py` (Python, used for training/evaluation and the FastAPI service) and `web/pipeline.js` (browser, used by the deployed application). Repository modules are referenced below; no source code is reproduced here.

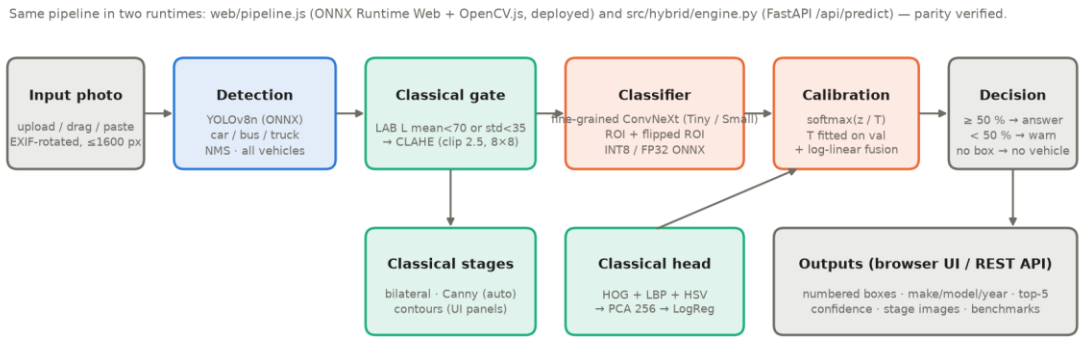


Figure 1: End-to-end architecture of Visionary

2.1 CLASSICAL ALGORITHMIC BASELINE

ALGORITHMS UTILISED

- **CLAHE** (contrast-limited adaptive histogram equalisation) applied to the **L channel of CIE-LAB** only, so that chromaticity is preserved (clip limit 2.5, 8 × 8 tiles) — `src/classical/preprocessing.py`.
- **Bilateral filtering** (d = 9, σ_colour = σ_space = 75) for edge-preserving denoising.
- **Canny edge detection** after a 3 × 3 Gaussian blur, followed by morphological closing and external **contour extraction** (area ≥ 50 px) — `src/classical/edges.py`.

- **Descriptors** — HOG (9 orientations, 16 × 16 cells on a 128 × 128 grey crop) for shape, uniform LBP (P = 24, R = 3) for texture and an 8 × 8 × 8 HSV histogram for colour, 2,302 dimensions in total — `src/hybrid/classical_head.py`.
- **Classical classifier** — standardisation → PCA (256 components) → multinomial logistic regression (C = 0.01, selected on the validation split).

ADAPTIVE SELECTION LOGIC

Parameters are computed from each image rather than fixed:

- **Illumination gate:** the vehicle crop is converted to LAB; if the mean lightness is below 70 or its standard deviation is below 35 (out of 255), CLAHE is applied before classification; otherwise the crop is used unchanged.
- **Canny thresholds** are derived from the median grey level m of the blurred crop: lower = 0.67·m, upper = 1.33·m, widened by ±30 if they collapse on flat images.
- **Region of interest** is the detector box padded by 5 % on each side; the primary vehicle is the box with the largest area × confidence.
- **Multi-vehicle selection:** every other box at least 48 px and 1 % of the frame is classified; duplicates (IoU > 0.5, or more than 80 % inside a kept box) are removed; at most six vehicles.

2.2 DEEP LEARNING PIPELINE

MODEL ARCHITECTURE

Two variants of **ConvNeXt** (Liu et al., 2022), pretrained on ImageNet-22k (timm), with a new 196-way linear head: **Visionary Pro** — ConvNeXt-Small at 320 × 320 px (49.61 M parameters) — and **Visionary Lite** — ConvNeXt-Tiny at 224 × 224 px (27.97 M parameters). ConvNeXt was selected because it offers transformer-level transfer accuracy with a convolutional design that is fast on CPUs, quantises well and exports cleanly to ONNX for in-browser execution. Vehicle localisation uses **YOLOv8n** (Jocher et al., 2023), COCO-pretrained, restricted to the car, bus and truck classes.

LOSS FUNCTIONS & TRAINING

- **Data:** Stanford Cars (Krause et al., 2013) — 8,144 training and 8,041 test images. A stratified 10 % of the training set (788 images) forms the validation split; the official test split is used only for final reporting.

- **Training input:** ground-truth box crops with random 0–15 % padding per side, so the classifier sees imperfect, detector-like crops; 15 % of samples use the full frame so it degrades gracefully if detection fails.
- **Augmentation:** horizontal flip, TrivialAugment and Random Erasing (a proxy for occlusion).
- **Loss:** cross-entropy with label smoothing 0.1; AdamW (lr 2e-4, 5× for the head, weight decay 0.05), one-epoch warm-up then cosine decay, gradient clipping, mixed precision, 15 epochs.
- **Hardware:** a single laptop RTX 3050 (6 GB): ≈ 47 s per epoch for Lite and ≈ 150 s for Pro.
- **Calibration:** the best epoch is chosen on validation; a softmax temperature fitted on validation (T = 0.6 for Pro, 0.55 for Lite) corrects the under-confidence caused by label smoothing.

2.3 HYBRID INTEGRATION & ADAPTIVE MECHANISM

FUSION STRATEGY

- **Detection → classification:** the classical ROI extraction feeds the network a tight crop of each vehicle, focusing capacity on the discriminative parts.
- **Illumination prior → network input:** the LAB brightness/contrast statistics decide whether CLAHE is applied before the network sees the crop.
- **Classical descriptors → late fusion:** a log-linear (product-of-experts) fusion log p ∝ log p_CNN + w·log p_classical, with w and a re-calibration temperature selected on the validation split. Unlike a weighted average, it can only re-rank classes the network already finds plausible, so it never flattens the distribution.
- **Test-time augmentation:** the primary vehicle is classified from the crop and its mirror image and the probabilities are averaged.

DECISION LOGIC

After calibration, the system answers when the top-1 probability is at least 50 %, shows the answer as a **low-confidence best guess** with the top-5 alternatives when it is lower, and reports **"no vehicle detected"** when there is no detector box and the classifier is not confident. Inputs are validated (empty files, non-images, images under 32 px or over 15 MB are rejected with a clear message).

An important, honestly reported outcome: once a data leak was removed (the classical head had been refitted on train + validation, inflating its validation scores), validation tuning selected a classical fusion weight of 0 for Pro and 0.1 for Lite. The classical branch therefore contributes through the ROI and the illumination gate rather than through late fusion; this is analysed in the ablation (Section 3.3).

3. EXPERIMENTAL BENCHMARKS & EVALUATION

All results are on the official Stanford Cars test split (8,041 images never used for training, early stopping, calibration or fusion tuning) and are produced by `scripts/evaluate.py`; the complete metric file is `eval_P5_T20.md`. Latency is end-to-end per image on a CPU limited to two threads (median of 40 images), matching a free two-vCPU hosting tier.

3.1 QUANTITATIVE BENCHMARKS

Table 3: Classical vs deep learning vs hybrid on the test set (8,041 images)

Approach / Model Variant	**Top-1 (%) ↑**	**Top-5 (%) ↑**	**Size (MB) ↓**	**Latency (ms / image) ↓**
Classical baseline (HOG + LBP + HSV → PCA → LogReg)	19.02	41.49	2.3	18.7
Deep learning only (ConvNeXt-Small, full image)	94.03	99.28	94.7	248.2
Proposed hybrid — Pro (FP32)	**94.73**	**99.3**	198.0	537.8
Proposed hybrid — Pro, INT8*	94.44	99.44	62.8	287.2
Proposed hybrid — Lite (ConvNeXt-Tiny, FP32)	93.99	99.08	115.4	206.3

* INT8 measured on a class-stratified subset of 1,960 test images; on the same images FP32 scores 94.44 % and 99.9 % of predictions are identical. Size includes the detector and classical head.

Table 4: Localisation, detection, calibration and implementation-parity metrics

Metric	**Value**
Mean / median IoU (primary YOLO box vs ground truth, 8,000 boxes)	0.930 / 0.952
Images with IoU ≥ 0.5	98.76 %
Vehicle detection rate	99.03 %
mAP@0.50 / mAP@0.50:0.95	0.777 / 0.713
Expected calibration error before → after temperature scaling	16.8 % → 1.9 %
Accuracy when confident (≥ 50 %) / when flagged (< 50 %)	95.72 % / 39.44 %
Low-confidence flag rate	1.77 %
Browser vs Python implementation (same 300 test images)	93.33 % vs 93.33 %; 98.0 % identical

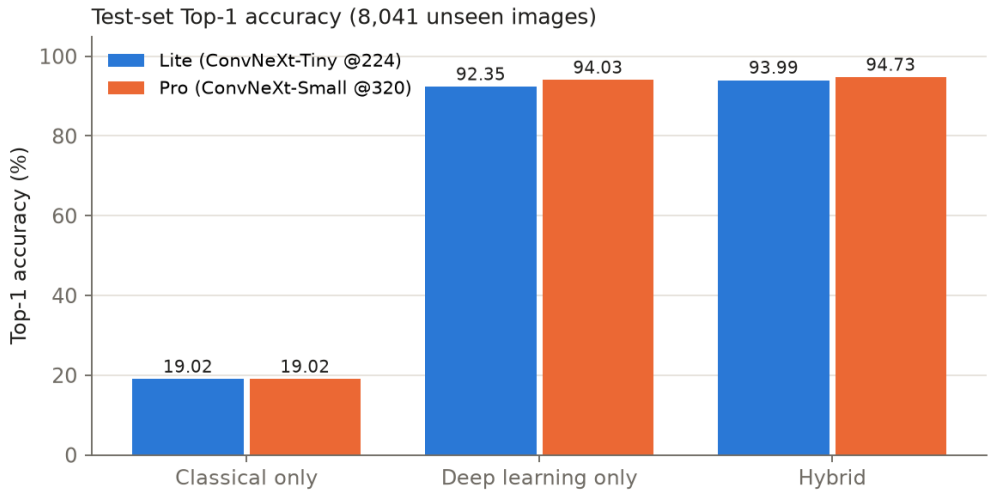


Figure 2: Top-1 accuracy of classical, deep-learning and hybrid approaches for both model variants

ANALYSIS

Hand-crafted features alone reach 19.02 % — far above chance (0.5 %) but unable to separate 196 look-alike classes. Fine-tuned deep features raise accuracy to 94.03 %, and the hybrid pipeline adds a further 0.70 points for Pro (+1.64 for Lite) at the cost of running the detector. The larger model is more accurate but about 2.6× slower; INT8 quantisation recovers most of the speed with no accuracy loss, which is why the browser runs INT8 models and defaults to Lite for phones.

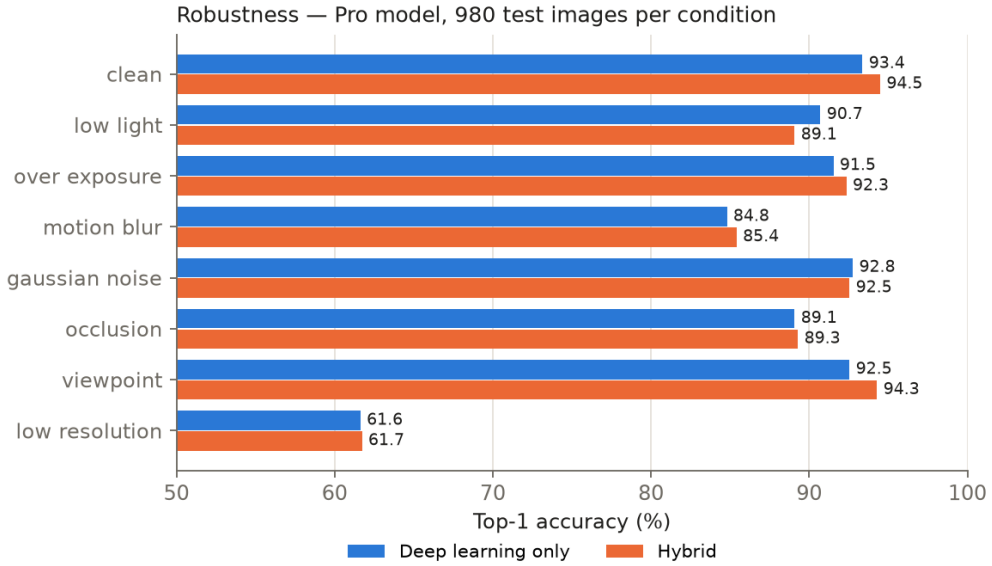


Figure 3: Robustness under seven synthetic corruptions (Pro model)

Under corruptions the hybrid is ahead in five of seven conditions, most clearly for viewpoint changes, over-exposure and motion blur. For the Pro model it is slightly

behind deep-learning-only under low light and noise, and both collapse at very low resolution (Section 4).

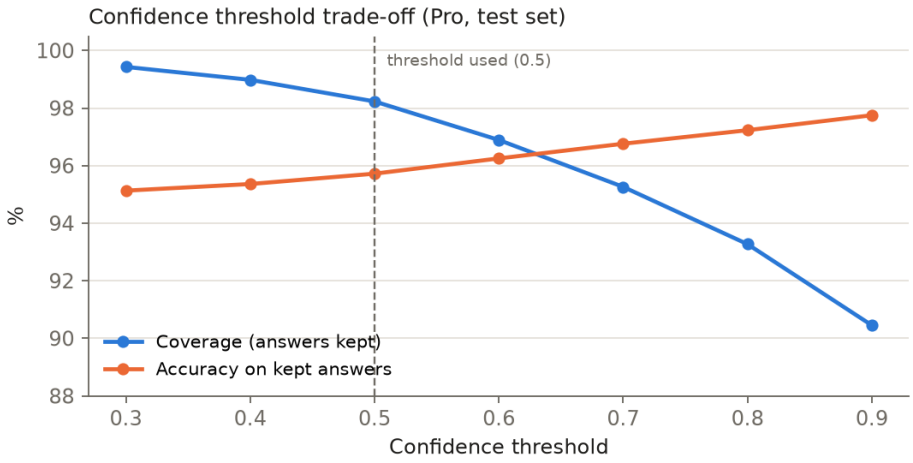


Figure 4: Confidence threshold trade-off between coverage and accuracy

3.2 VISUAL OUTPUTS & INTERMEDIATE REPRESENTATIONS

Figure 5 shows outputs of the deployed pipeline on unseen test images: a single car, a dark studio image for which the illumination gate switched CLAHE on, and a dealership scene in which every vehicle is detected and numbered. Figure 6 shows the intermediate classical representations displayed in the application for the dark image.



Figure 5: Outputs on unseen test images — single vehicle, dark image (CLAHE applied), several vehicles



Figure 6: Intermediate states for the dark image: ROI, CLAHE, bilateral filter, Canny edges, contours

3.3 ABLATION STUDY

Each configuration adds one component to the one above (A and B are independent baselines), evaluated on the same cached detections so that differences are attributable to the component alone.

Table 5: Ablation study on the test set (Top-1 %)

Configuration	Lite Top-1	Δ	Pro Top-1	Δ
A. Classical only (YOLO ROI + HOG/LBP/HSV + LogReg)	19.02	–	19.02	–
B. Deep learning only (full image)	92.35	–	94.03	–
C. B + adaptive CLAHE gate	92.29	-0.06	94.08	+0.05
D. YOLO ROI crop + deep learning	94.01	+1.72	94.58	+0.50
E. D + horizontal-flip TTA	94.17	+0.16	94.73	+0.15
F. E + full-frame context fusion	94.17	+0.00	94.73	+0.00
G. F + adaptive CLAHE gate	94.17	+0.00	94.73	+0.00
H. G + classical late fusion (full hybrid)	93.99	-0.18	94.73	+0.00

- **Detector ROI** is the largest single gain (+1.66 for Lite, +0.55 for Pro): the stronger model needs less help.
- **Flip test-time augmentation** adds a consistent +0.15 points.
- **The adaptive CLAHE gate** is neutral on clean test images (it rarely fires) — its value appears under low-light and flat images, and it is never harmful on good images, unlike always-on enhancement.
- **Full-frame context and classical late fusion** received zero weight from validation tuning; adding them does not change test accuracy. We report this rather than tune on the test set.

4. FAILURE MODES & LIMITATIONS

The Pro hybrid makes 424 errors on 8,041 test images (5.3 %). Mining the confusion matrix shows that they cluster into explainable groups (full analysis in `docs/failure_cases.md`).

Table 6: Most frequent confusions of the Pro model on the test set

Most frequent confusion (true → predicted)	Errors
Audi S5 Coupe 2012 → Audi A5 Coupe 2012	14
Chevrolet Express Van 2007 → Chevrolet Express Cargo Van 2007	13
Audi TTS Coupe 2012 → Audi TT Hatchback 2011	12
Audi V8 Sedan 1994 → Audi 100 Sedan 1994	12
Bentley Continental GT Coupe 2012 → Bentley Continental GT Coupe 2007	10
Dodge Caliber Wagon 2012 → Dodge Caliber Wagon 2007	10



Figure 7: Observed failure cases: facelift year, roof-up convertible, same-maker sedan

4.1 OBSERVED FAILURE CASES

- **Facelift years and trims** — Bentley Continental GT 2012 → 2007, Dodge Caliber 2012 ↔ 2007, Audi S5 → A5, Audi TTS ↔ TT. Errors barely change between Lite and Pro, so more capacity alone does not solve them.
- **Body-style ambiguity and same-maker look-alikes** — a Ferrari 458 coupe predicted as the convertible (roof closed), a Toyota Corolla predicted as a Camry; these occur at almost 100 % confidence, so calibration cannot flag them.
- **Very low resolution** — when the image is reduced to 80 px, accuracy falls to about 62 % and the detector misses about 10 % of vehicles; cropping a tiny, blurry box removes useful context.
- **Out-of-distribution cars** — the dataset covers mostly US-market models from 1991–2012, so newer or regional cars are mapped to the nearest known class; usually with low confidence, but not always.

4.2 ROOT CAUSE & MITIGATION

- **Root cause (facelifts, trims):** the discriminative cue occupies a few pixels (headlight internals, badge, bumper intakes) and each class has about 41 training images, so the network relies on the shared silhouette. **Mitigation:** part-based attention or higher-resolution crops of the front and rear, hierarchical make → model → trim prediction so the system can answer “Audi A5/S5 family” when unsure.
- **Root cause (confident errors):** a closed-set softmax must choose one of 196 labels and is sure when two classes look identical. **Mitigation:** open-set scoring on the embedding (energy or Mahalanobis distance) to abstain on unfamiliar inputs; the top-5 list already shows the correct sibling as the runner-up.

- **Root cause (low resolution):** the detector box shrinks below the network's effective receptive detail. **Mitigation:** fall back to full-frame classification (the pipeline's full-frame weight) when the box is smaller than about 64 px.
- **Dataset limitation:** collect recent and Indian-market vehicles and fine-tune; the threshold analysis shows that keeping the 50 % gate removes the least reliable 1.77 % of answers.

4.3 LIMITATIONS AND ETHICS

The system does not read licence plates, recognise faces, track vehicles or store images; the browser application never uploads photographs. Predictions are presented with confidence and alternatives and are intended for educational and analytic use, not for enforcement decisions.

5. DEPLOYMENT & QUICKSTART GUIDE

5.1 QUICKSTART COMMANDS

```
# 1. Clone the repository and install dependencies
git clone https://github.com/chakradhar1778/Visionary.git
cd Visionary
pip install -r requirements-deploy.txt
pip install torch torchvision --index-url
https://download.pytorch.org/whl/cpu
pip install --no-deps ultralytics ultralytics-thop

# 2. Run the application (web UI + REST API) on http://localhost:7860
uvicorn app.main:app --port 7860

# or: one command with Docker
docker build -t visionary . && docker run -p 7860:7860 visionary

# 3. Reproduce training and every reported metric
python scripts/prepare_data.py
python scripts/train.py --backbone convnext_small.fb_in22k_ft_in1k --img 320
--name convnext_small_320
python scripts/evaluate.py --ckpt weights/convnext_small_320.pt
python scripts/write_report.py --default convnext_small_320 --extra
convnext_tiny_224
```

5.2 DEPLOYMENT FORMAT

- **In-browser web application** (primary): `web/` runs YOLOv8n and the INT8 ConvNeXt with ONNX Runtime Web (multi-threaded WebAssembly) and the classical stages with OpenCV.js; hosted free on a Hugging Face static Space at https://visionalpha-visionary.static.hf.space. Models (≈ 66 MB) are cached after

the first visit; inference takes about 2–3 s per photo in a laptop browser, and photos never leave the device.

- **FastAPI service / Docker container:** `app/main.py` serves the same web application with the cross-origin-isolation headers and exposes `POST /api/predict`, `GET /api/models` and `GET /api/benchmarks`; it runs the ONNX Runtime FP32 or INT8 models on CPU (≈ 0.14–0.54 s per image).
- **Model formats:** PyTorch checkpoints for training; ONNX FP32 and dynamic-INT8 for deployment.
- **Quality assurance:** 49 automated tests (classical CV, detection, classifier, API including bad input and multi-vehicle images) and a JavaScript-vs-Python parity check (`scripts/check_web_parity.py`).

STUDENT SUBMISSION CHECKLIST

☐ Working GitHub repository link included with an updated README.md.

☑ No raw code pasted into this report.

☑ All table metrics match test runs reproducible via the repository (`scripts/evaluate.py`, `eval_P5_T20.md`).

☐ All team member details accurately recorded (team name, guide and signatures to be completed).

REFERENCES

Canny, J. (1986). A computational approach to edge detection. IEEE Transactions on Pattern Analysis and Machine Intelligence, 8(6), 679–698.

Dalal, N., & Triggs, B. (2005). Histograms of oriented gradients for human detection. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (Vol. 1, pp. 886–893).

Guo, C., Pleiss, G., Sun, Y., & Weinberger, K. Q. (2017). On calibration of modern neural networks. In Proceedings of the 34th International Conference on Machine Learning (pp. 1321–1330). PMLR.

Jocher, G., Chaurasia, A., & Qiu, J. (2023). Ultralytics YOLOv8 (Version 8) [Computer software]. https://github.com/ultralytics/ultralytics

Krause, J., Stark, M., Deng, J., & Fei-Fei, L. (2013). 3D object representations for fine-grained categorization. In Proceedings of the IEEE International Conference on Computer Vision Workshops (pp. 554–561).

Liu, Z., Mao, H., Wu, C.-Y., Feichtenhofer, C., Darrell, T., & Xie, S. (2022). A ConvNet for the 2020s. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (pp. 11976–11986).

Microsoft. (2024). ONNX Runtime and ONNX Runtime Web [Computer software]. https://onnxruntime.ai

Müller, S. G., & Hutter, F. (2021). TrivialAugment: Tuning-free yet state-of-the-art data augmentation. In Proceedings of the IEEE/CVF International Conference on Computer Vision (pp. 774–782).

Ojala, T., Pietikäinen, M., & Mäenpää, T. (2002). Multiresolution gray-scale and rotation invariant texture classification with local binary patterns. IEEE Transactions on Pattern Analysis and Machine Intelligence, 24(7), 971–987.

Tomasi, C., & Manduchi, R. (1998). Bilateral filtering for gray and color images. In Proceedings of the Sixth International Conference on Computer Vision (pp. 839–846).

Zhong, Z., Zheng, L., Kang, G., Li, S., & Yang, Y. (2020). Random erasing data augmentation. In Proceedings of the AAAI Conference on Artificial Intelligence, 34(7), 13001–13008.

Zuiderveld, K. (1994). Contrast limited adaptive histogram equalization. In P. S. Heckbert (Ed.), Graphics Gems IV (pp. 474–485). Academic Press.